Submitted by: Yuriy Rusanov – 340915149, Elad Orpaz - 206216517

Deep Learning

Exercise 4

1. Data

The dataset used for this assignment is the "Adult" dataset, which is provided in ARFF format. This dataset contains a mix of numerical and categorical features, with the target feature being "income."

Train-Test Split: The data was split into training and testing sets using an 80% / 20% ratio. To ensure that the class proportions of the target feature ("income") were maintained in both the training and testing sets, a stratified split was employed.

Preprocessing: Several preprocessing steps were applied to prepare the data for the generative models:

1. Feature Identification: Numerical and categorical features were identified based on the ARFF header information.
    - Numerical Features: 'age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week'.
    - Categorical Features: 'workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country'.
2. Handling Missing Values:
    - For numerical features, missing values (if any) were imputed using the mean of the respective column.
    - For categorical features, missing values in 'workclass', 'occupation', and 'native-country' were imputed using the mode of the respective column.
3. Target Variable Encoding: The target variable "income" was encoded using LabelEncoder into numerical values (0 and 1), representing the classes: ['<=50K' '>50K'].
4. Feature Transformation (within each experiment's training phase):
    - Numerical Features: MinMaxScaler was used to normalize numerical features to a range between 0 and 1.
    - Categorical Features: OneHotEncoder was used to convert categorical features into a one-hot numerical representation.
    - These transformations were applied using ColumnTransformer, which was fit only on the training data for each experiment to prevent data leakage from the test set. The test set was then transformed using the fitted preprocessor.

2. GAN (Generative Adversarial Network)

Implementation: A GAN was implemented to generate synthetic tabular data. The GAN consists of two main components: a Generator and a Discriminator.

Generator:

The Generator receives random noise (latent vector) as input and aims to produce synthetic data samples that resemble the real data distribution.

Architecture:

- An initial shared trunk of two fully connected layers (Linear) with LeakyReLU activation and BatchNorm1d for stabilization.
- Two separate heads emerge from the trunk:
    - **Numerical Head:** A linear layer followed by a Sigmoid activation function was chosen because the numerical features in the training data were normalized to a range of 0 to 1 using MinMaxScaler. The Sigmoid function naturally outputs values in this range, making it suitable for generating these scaled numerical features.
    - **Categorical Head:** A linear layer outputs logits for the one-hot encoded categorical features. Simply using a Softmax function would produce probabilities for each category, not discrete one-hot vectors (e.g. [1,0,0]). While an argmax operation could convert these probabilities to one-hot vectors, argmax is not differentiable, which is problematic for backpropagation. Therefore, Gumbel-Softmax with hard=True was applied during the forward pass. This technique allows for differentiable sampling from a categorical distribution, producing discrete one-hot vectors suitable for training with backpropagation.

Discriminator:

The Discriminator (Discriminator class) receives data samples (either real or synthetic) and attempts to differentiate between them.

Architecture:

- A sequence of three fully connected layers.
- LeakyReLU activation functions are used after the first two linear layers.
- Dropout layers (with p=0.3) are included after the LeakyReLU activations for regularization, helping to prevent overfitting.
- The final layer uses a Sigmoid activation function to output a probability score indicating whether the input sample is real (closer to 1) or synthetic (closer to 0).

We initially tested a discriminator that processed concatenated real and fake sample pairs but found this approach to be less stable. Essentially, it's the same task. Even when processing a pair, the discriminator only needs to learn how to identify **one** of the samples. For instance, if it can spot that one sample is fake, it automatically knows the other is real, since their labels in the pair are always the opposite of each other. Since the challenge boils down to the same job of "spotting the fake," we chose the more computationally efficient and stable single-sample architecture.

Training:

- Hyperparameters:
    - Epochs: 100
    - Batch Size: 128
    - Learning Rate (Generator - LR_G): 0.00002
    - Learning Rate (Discriminator - LR_D): 0.00001
    - Latent Dimension (LATENT_DIM): 64 (This is the size of the random noise vector input to the generator)
    - Optimizer: Adam optimizer was used for both the generator and discriminator.

  Rationale for Learning Rates: The generator's learning rate (LR_G) was set slightly higher than the discriminator's learning rate (LR_D). This is a common practice in GAN training. If the discriminator learns too quickly and becomes too strong, it can easily distinguish real from fake samples, providing very little information for the generator to learn. The chosen rates aim for a stable training dynamic.

- Loss Function: Binary Cross-Entropy loss (nn.BCELoss) was used.

  Rationale for BCE Loss: BCE loss is a standard choice for GANs when the discriminator's final layer uses a sigmoid activation function, as it outputs a probability between 0 (fake) and 1 (real).

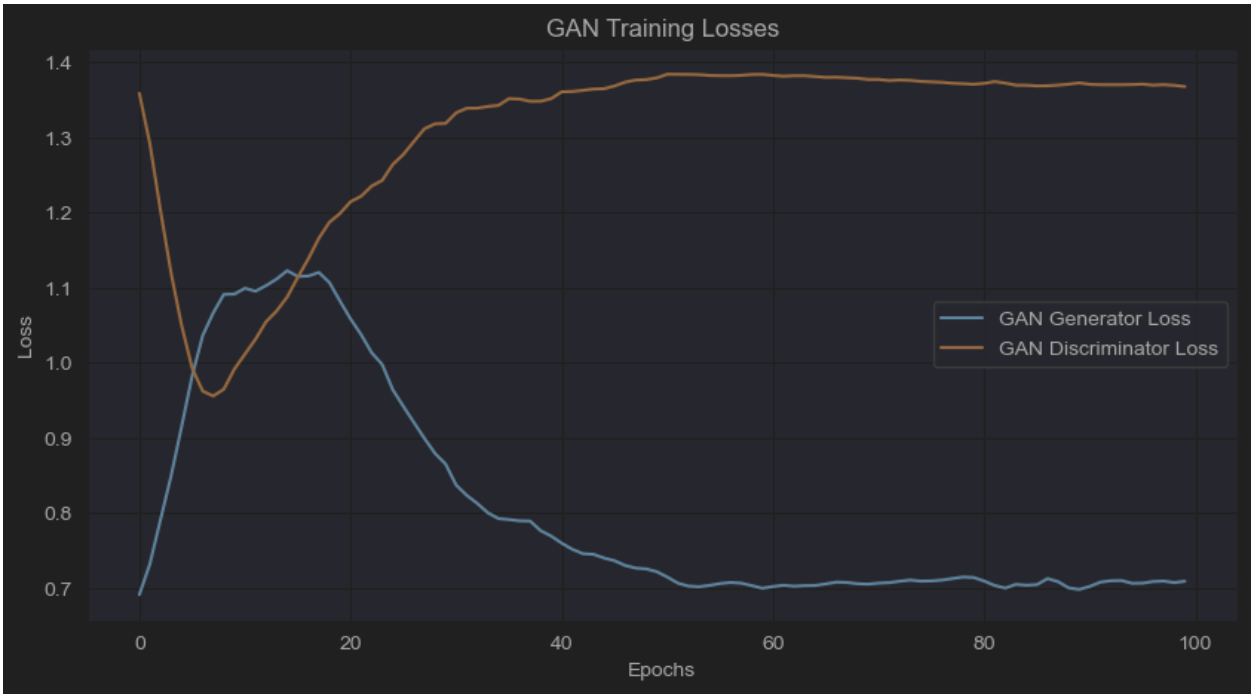
- Process:

    - The Discriminator is trained by feeding it a batch of real samples (labeled as real) and a batch of synthetic samples generated by the Generator (labeled as fake). The discriminator's weights are updated to minimize its classification error.

    - The Generator is trained by feeding random noise to produce synthetic samples. These synthetic samples are then passed through the Discriminator. The Generator's weights are updated to maximize the Discriminator's error for these fake samples (i.e., to make the Discriminator classify them as real). During generator training, the discriminator's weights are kept frozen.

- Model Saving: The state dictionaries of the generator and discriminator models were saved whenever the generator's average loss for an epoch improved upon the best generator loss recorded so far, starting from epoch 11. This strategy aims to retain the model that was best at fooling the discriminator during training.

Losses for seed 42:



Interpretation of BCE Loss in GANs:

- **Generator Loss:** The generator aims to make the discriminator output 1 for fake samples. If the generator perfectly fools the discriminator ($D(G(z)) = 0.5$, meaning the discriminator is unsure), the generator's loss would ideally be $-log(0.5) \approx 0.693$.
- **Discriminator Loss:** The discriminator's loss is the sum of two BCE terms: one for real data (where it tries to output 1) and one for fake data (where it tries to output 0). Ideally, if the discriminator is performing at random (i.e., it cannot distinguish real from fake, $P(real) = 0.5, P(fake) = 0.5$), its loss would be approximately $-(log(0.5) + log(0.5)) = -2log(0.5) \approx 1.386$.

The best iteration loss for:

- Generator – 0.6981
- Discriminator – 1.3728

The GAN training graph and loss values indicate a successful training process. The generator's loss stabilized near the ideal value of ~0.693, signifying it effectively learned to produce samples that challenge the discriminator. Concurrently, the discriminator's loss approached its theoretical equilibrium of ~1.386 (actual: 1.3728), where it performs similarly to random guessing. This convergence suggests a good balance between the generator and discriminator, with the generator producing realistic outputs.

2. cGAN (Conditional Generative Adversarial Network)

Implementation: A GAN was implemented to generate synthetic tabular data. The GAN consists of two main components: a Generator and a Discriminator.

Generator:

The Conditional Generator is similar to the GAN generator but additionally receives the conditional information (the desired label) as input.

Architecture:

- The input to the generator is a concatenation of the random noise (latent vector) and a one-hot encoded representation of the desired class label.
- The rest of the architecture (shared trunk, numerical head with Sigmoid, categorical head with Gumbel-Softmax) is similar to the GAN generator, with the initial linear layer of the trunk adjusted to accept the combined size of the latent vector and the one-hot label vector.

Discriminator:

The Conditional Discriminator also receives the conditional information along with the data sample (real or synthetic).

Architecture:

- The input to the discriminator is a concatenation of the data sample (features) and the one-hot encoded class label corresponding to that sample.
- The subsequent architecture (sequence of linear layers, LeakyReLU, Dropout, and final Sigmoid output) is similar to the GAN discriminator, with the initial linear layer adjusted for the combined input size.

For real samples, the true label of the sample is provided. For synthetic samples, the label that the generator was conditioned on is provided.

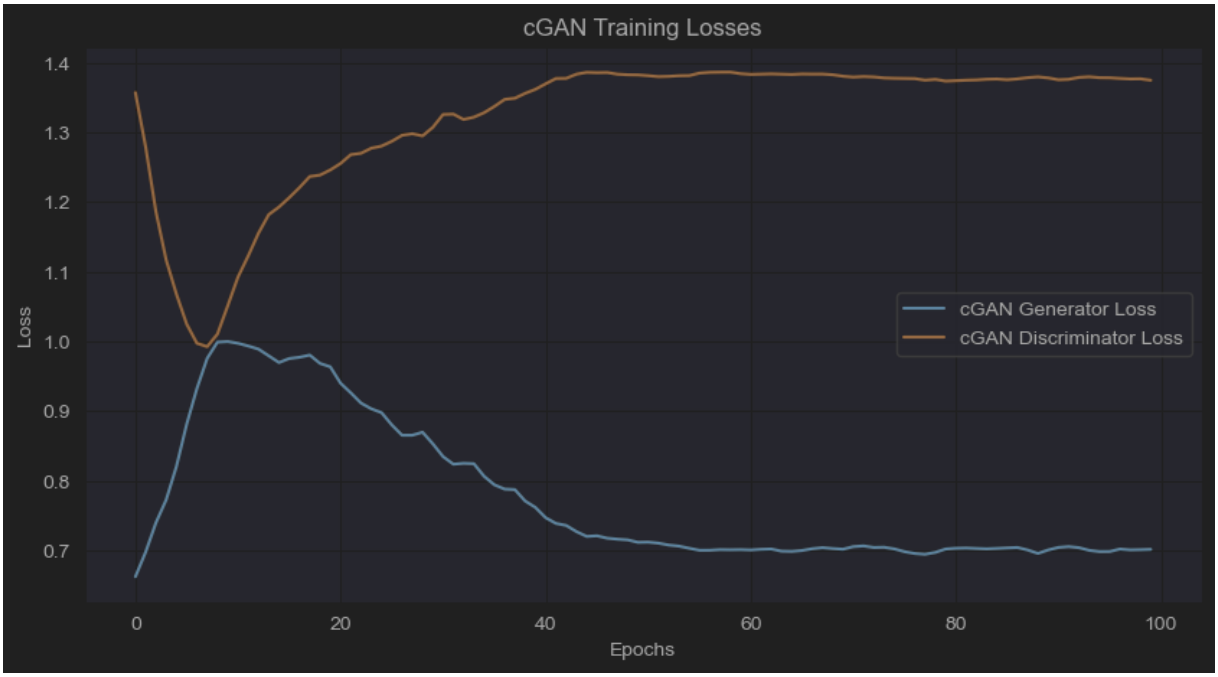
Training:

- Hyperparameters:
    - Epochs: 100
    - Batch Size: 128
    - Learning Rate (Generator - LR_G): 0.00002
    - Learning Rate (Discriminator - LR_D): 0.00001
    - Latent Dimension (LATENT_DIM): 64
    - Optimizer: Adam optimizer was used for both the generator and discriminator.
- Loss Function: Binary Cross-Entropy loss (nn.BCELoss) was used.

- Process:
    - The **Conditional Discriminator** is trained by feeding it:
        - Real samples concatenated with their true one-hot encoded labels (labeled as real).
        - Synthetic samples (generated by the Conditional Generator based on specific one-hot encoded labels) concatenated with those same one-hot encoded labels (labeled as fake).

    - The **Conditional Generator** is trained by generating synthetic samples based on random noise and a batch of one-hot encoded labels (mirroring the label distribution of the real data batch). These synthetic samples, concatenated with their conditioning labels, are passed to the Discriminator. The Generator's weights are updated to make the Discriminator classify these conditional fakes as real.

- Model Saving**:** Similar to the GAN, the best performing cGAN generator and discriminator (based on generator loss after epoch 10) were saved.
- Losses for seed 42:



The best iteration loss for:

- Generator – 0.6945
- Discriminator – 1.3745

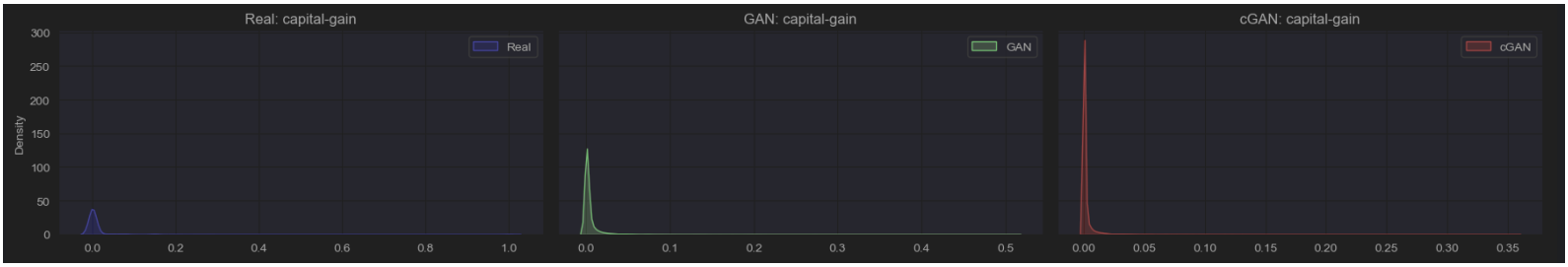
The cGAN training achieves similar results to the standard GAN, stabilizing near their theoretical equilibrium values. This indicates the cGAN effectively learned to generate realistic conditional samples, achieving a good balance where the discriminator struggled to differentiate them from real data.

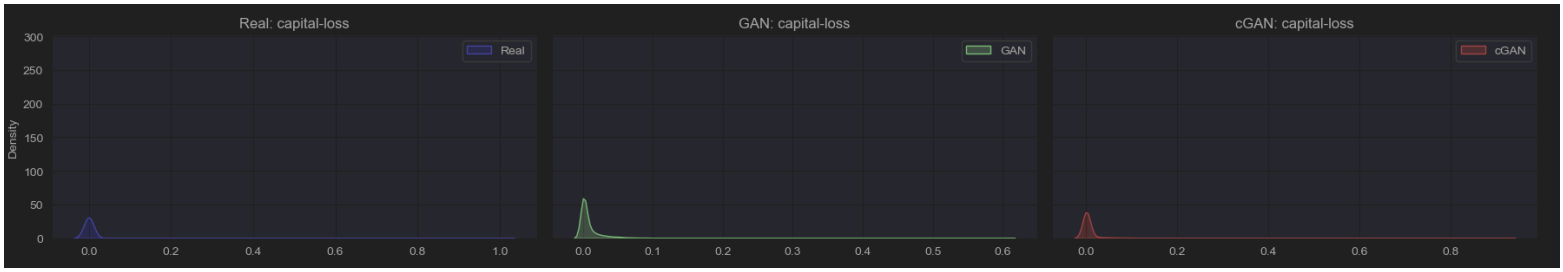
4. Comparison of Data Distribution

After training, both the GAN and cGAN models were used to generate synthetic datasets.

- For the GAN, a synthetic dataset equal in size to the training set was generated.
- For the cGAN, a synthetic dataset was generated with requested label ratios identical to those of the original training set.

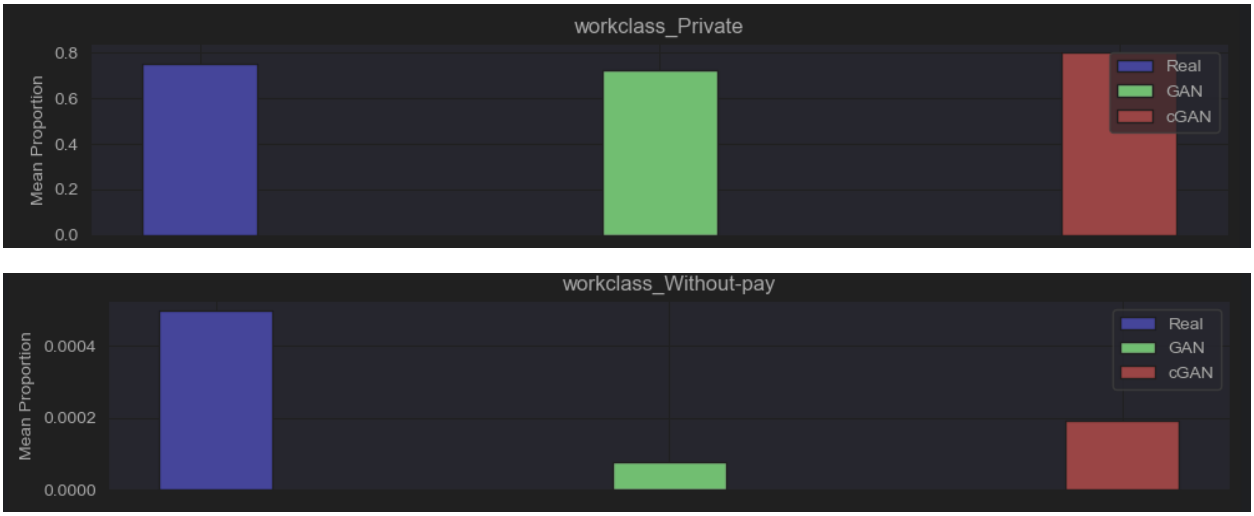
Comparison of some numerical features distributions:





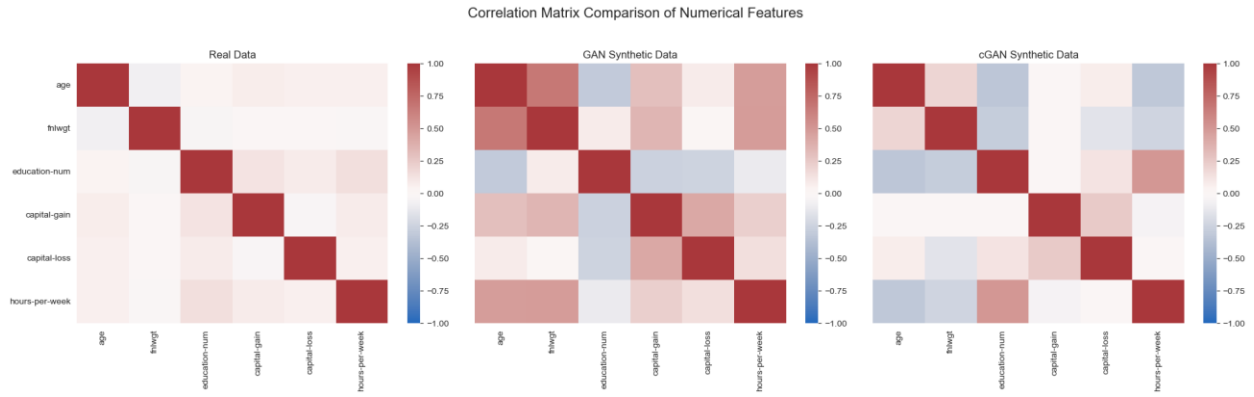
For numerical features like 'capital-gain' and 'capital-loss', both GAN and cGAN correctly identified them as mostly zero. However, their generated distributions show taller, narrower peaks at zero, suggesting they might overemphasize zero values and under-represent the sparse, non-zero tails compared to real data.

Comparison of some categorical features distributions:



For categorical features, both models effectively replicated the high proportion of the common 'workclass_Private' category. For the rare 'workclass_Without-pay' category, the standard GAN significantly under-represented it. The cGAN performed better, getting closer to the real proportion, but still found it challenging to perfectly match this very infrequent category.

In summary, while both models capture dominant patterns, they face challenges with the nuances of sparse distributions and accurately replicating very rare categories, with the cGAN showing some advantages in categorical feature generation.



In the real data, most features are weakly correlated, with a few stronger patterns like the ones between education-num and hours-per-week. The GAN-generated data doesn’t preserve this structure very well and adds some correlations that don’t exist in the real data. On the other hand, the cGAN does a better job of keeping the original relationships between features. Although it's not perfect, the cGAN seems to generate data that is more similar to real data in terms of how the features relate to each other.

5. Results

The performance of the GAN and cGAN models was evaluated using detection and efficacy metrics. These evaluations were conducted for three separate experiments, each using a different random seed for the initial train-test split and for any random processes within the evaluation functions themselves. The average results across these three experiments will be reported.

Detection Metric

The detection metric aims to assess how distinguishable synthetic data is from real data. A lower score (AUC closer to 0.5) is better, indicating that the synthetic samples are highly similar to real samples and thus difficult for a classifier to distinguish.

Process:

- The original training set (80% of the full dataset) and the generated synthetic dataset (equal in size to the training set) were used.
- Both datasets were split into four folds (K_FOLDS_DETECTION = 4).
- For each of the four iterations:
    - A Random Forest (RF) classifier and a Logistic Regression (LR) classifier were trained on a combined dataset of three folds of real data and three folds of synthetic data. The target label for this classification task was 0 for real samples and 1 for synthetic samples.
    - The trained classifiers were then evaluated on a test set composed of the remaining one fold of real data and one fold of synthetic data.
- The Area Under the ROC Curve (AUC) was calculated for both RF and LR detectors in each iteration.
- The average AUC across the four iterations is reported for both detectors.

Efficacy Metric

The efficacy metric aims to determine whether synthetic data is a useful substitute for the real data in a machine learning task. A higher score (closer to 1) is better, indicating that a model trained on synthetic data performs comparably to a model trained on real data when evaluated on a real test set.

Process:

- **Real Data Performance:** A Random Forest (RF) model was trained on the original training set (X_train_processed, y_train) and evaluated on the original test set (X_test_processed, y_test). The AUC score (AUC_real) was obtained.
- **Synthetic Data Performance:**
    - For the **GAN**, labels for the synthetic data were predicted using a Random Forest classifier (label_predictor_for_gan) trained on the real processed training data and its true labels. This step is necessary because the standard GAN does not inherently generate labels alongside features for the efficacy task where labels are needed for training the downstream Random Forest.
    - For the **cGAN**, the labels generated alongside the features (synthetic_labels_cgan) were used directly.
    - An RF model was then trained on the respective synthetic dataset (GAN-generated features with predicted labels, or cGAN-generated features with their generated labels) and evaluated on the original test set (X_test_processed, y_test). The AUC score (AUC_synthetic) was obtained.
- **Efficacy Score Calculation:** The efficacy was calculated as AUC_synthetic / AUC_real.

Results Summary:

		GAN	cGAN
	Detection RF (AUC)	1	1
	Detection LR (AUC)	0.599	0.6467
	Efficacy (AUC Real)	0.8862	0.8862
	Efficacy (AUC Synthetic)	0.8366	0.8288
Seed 42	Efficacy (Ratio)	0.9440	0.9352
	Detection RF (AUC)	1	1
	Detection LR (AUC)	0.7283	0.6214
	Efficacy (AUC Real)	0.8742	0.8742
	Efficacy (AUC Synthetic)	0.8440	0.7597
Seed 43	Efficacy (Ratio)	0.9655	0.869
	Detection RF (AUC)	1	1
	Detection LR (AUC)	0.6454	0.6415
	Efficacy (AUC Real)	0.8733	0.8733
	Efficacy (AUC Synthetic)	0.8420	0.7976
Seed 44	Efficacy (Ratio)	0.9642	0.9133

Average	GAN	cGAN
Detection RF (AUC)	1	1
Detection LR (AUC)	0.6576	0.6365
Efficacy (Ratio)	0.9579	0.9058

Detection Metric Analysis:

Random Forest (RF) Detection:

- Both the **GAN** and **cGAN** achieved an average **Detection RF (AUC) of 1.0**. This indicates that the Random Forest classifier was able to perfectly distinguish synthetic samples from real samples for both models. This suggests that, from the perspective of a relatively complex model like RF, the generated data from both GAN and cGAN still contains discernible differences from the real data.

Logistic Regression (LR) Detection:

  - The **GAN** had an average **Detection LR (AUC) of 0.6576**.
  - The **cGAN** had an average **Detection LR (AUC) of 0.6365**.
  - Both models performed significantly better (i.e., were harder to detect) with the simpler Logistic Regression classifier compared to the Random Forest. An AUC closer to 0.5 means the classifier is closer to random guessing.
  - The **cGAN's average AUC (0.6365) is slightly lower than the GAN's (0.6576)**, suggesting that the cGAN's output might be marginally more difficult for a linear classifier to distinguish from real data than the standard GAN's output. This hints at a slightly higher degree of linear realism in the cGAN samples.

While a powerful classifier (RF) can perfectly separate synthetic from real data for both models, a simpler linear classifier (LR) finds it more challenging, with the cGAN data being slightly harder to detect than GAN data.

Efficacy Metric Analysis:

GAN Efficacy:

- The average **Efficacy (Ratio) for the GAN is 0.9579**. This is a strong result, indicating that a Random Forest model trained on GAN-generated data (with labels predicted by another model) achieved approximately **95.79%** of the performance of a model trained on real data. This suggests high utility of the GAN-generated data for this specific task.

cGAN Efficacy:

- The average **Efficacy (Ratio) for the cGAN is 0.9058**. This means a Random Forest model trained on cGAN-generated data (using its directly generated conditional labels) achieved about **90.58%** of the performance of a model trained on real data. While still indicating good utility, this is noticeably lower than the GAN's efficacy.

In summary for efficacy: The synthetic data generated by the standard GAN proved to be a more effective substitute for real data in the downstream income prediction task compared to the data generated by the cGAN.